

Aula 11 - Web Services, SOAP, REST e desenvolvimento de APIs

TJSC 2026 - Analista de Sistemas - Banca FGV

Engenharia de Software e Desenvolvimento

Recorte da aula

Esta aula cobre os itens 1.13 Web Services padrões SOAP e REST e 1.14 Desenvolvimento de APIs, conforme a trilha deste chat. Não substitui as aulas de arquitetura, segurança ou linguagens; usa esses temas apenas quando necessários para entender web services e APIs em prova.

Padrão editorial

Material original, com teoria progressiva, exemplos de sistemas corporativos, diferenças entre prática real e cobrança FGV, 20 questões autorais com gabarito separado e comentários completos. Sem flashcards e sem blocos escuros, para preservar acessibilidade.

1. Mapa da aula e foco de prova

Nesta aula, o objetivo não é ensinar você a implementar do zero uma API em Quarkus ou Angular. Você já tem prática com APIs. O objetivo é transformar essa prática em acerto de prova FGV, isto é, dominar a linguagem conceitual que a banca usa quando compara Web Services, SOAP, REST, HTTP, JSON, XML, contrato de API e boas práticas de desenho de endpoints.

A FGV costuma cobrar esse tema com enunciados que misturam definição, cenário corporativo, afirmativas I, II e III, relação entre métodos HTTP, confusão entre REST e JSON, comparação entre SOAP e REST, e leitura de trechos de endpoint ou contrato. O erro mais comum do candidato experiente é responder pela prática do time, e não pela definição formal.

Ponto do edital/trilha	Como cai em prova	Risco de erro
Web Services	Definição, interoperabilidade, comunicação máquina-a-máquina e padrões de troca de mensagens	Achar que web service é sempre REST ou sempre HTTP
SOAP	Envelope XML, Header, Body, Fault, WSDL, contrato formal e uso de protocolos de transporte	Dizer que SOAP usa JSON ou que só funciona com HTTP
REST	Estilo arquitetural, recursos, representações, interface uniforme, stateless, cache, métodos HTTP	Chamar qualquer endpoint JSON de REST
Desenvolvimento de APIs	Contrato, recursos, versionamento, documentação, erros, paginação, status codes, idempotência	Confundir path/query, PUT/PATCH/POST, 401/403/404/409

2. Fontes e calibração FGV

Foram usadas como referência de conteúdo primário e de calibração: edital retificado do TJSC 2026; Manual Mestre de Calibração FGV do projeto; provas oficiais FGV correlatas; W3C para SOAP 1.2; RFC 9110 para semântica HTTP; RFC 5789 para PATCH; RFC 8259 para JSON; OpenAPI Specification 3.1.0; e referência clássica de Roy Fielding sobre REST.

O que a FGV costuma fazer
A banca raramente pede 'opinião de mercado'. Ela pede distinção precisa: SOAP é protocolo baseado em XML ; REST é estilo arquitetural ; API é interface; JSON é formato; HTTP é protocolo de aplicação; OpenAPI documenta APIs HTTP; WSDL descreve serviços SOAP .

3. Conceitos-base

3.1 API - Application Programming Interface

API - Application Programming Interface - é uma interface de programação de aplicações. Em termos simples, é um contrato pelo qual um software expõe funcionalidades ou dados para outro software. Nem toda API é web: uma biblioteca Java, uma interface TypeScript, um SDK, um driver JDBC ou um endpoint HTTP podem ser APIs em sentidos diferentes.

Em prova, quando o edital junta APIs com Web Services, SOAP e REST, a tendência é tratar API como interface de integração exposta via rede, normalmente usando HTTP - Hypertext Transfer Protocol - e payloads em JSON - JavaScript Object Notation - ou XML - Extensible Markup Language.

Prova x trabalho
No trabalho, você talvez diga 'chamar a API' para qualquer integração HTTP. Na prova, API é conceito amplo; Web Service é uma forma de disponibilizar serviço por rede; REST e SOAP são padrões/abordagens diferentes dentro desse universo.

3.2 Web Service

Web Service é um serviço de software acessível por rede, projetado para interoperabilidade entre aplicações. **A ideia central é permitir comunicação máquina-a-máquina, independentemente da linguagem usada em cada ponta.** Um sistema Java/Quarkus pode consumir serviço exposto por .NET, PHP, mainframe ou outro sistema, desde que ambos respeitem o contrato de comunicação.

O termo não deve ser reduzido a 'serviço REST'. Em provas FGV, Web Services aparecem associados a SOAP, REST, XML, JSON, WSDL, interoperabilidade, HTTP e consumo por sistemas heterogêneos.

3.3 Recurso, representação e contrato

Em uma API HTTP, especialmente quando desenhada com princípios REST, o recurso é a entidade lógica exposta pelo serviço, como processo, usuário, documento, certidão ou comarca. A representação é o modo como o estado desse recurso é transferido: JSON, XML, CSV ou outro formato. O contrato descreve endpoints, parâmetros, corpo de requisição, respostas, códigos de status e regras de erro.

```
GET /api/v1/processos/12345
Accept: application/json
```

Resposta: representação JSON do recurso processo 12345

4. SOAP - Simple Object Access Protocol

4.1 Ideia central

SOAP - Simple Object Access Protocol - é um protocolo de mensagens baseado em XML. A especificação SOAP 1.2 define um framework de mensagens com Envelope, Header, Body e Fault. A mensagem SOAP pode ser transportada por HTTP, mas o modelo é independente do protocolo de transporte, podendo ser associado a outros mecanismos, conforme binding.

A lógica de SOAP é fortemente contratual e orientada a operações. O cliente invoca uma operação descrita em contrato; o serviço recebe um envelope XML, processa o pedido e devolve outro envelope XML. É comum em sistemas legados, bancários, governamentais e integrações que exigem contrato formal rígido, segurança de mensagem ou aderência a padrões WS-*.

Elemento SOAP	Função
Envelope	Elemento raiz da mensagem SOAP. Delimita a mensagem.
Header	Opcional. Carrega metadados, como autenticação, roteamento, transação ou informações de segurança.
Body	Obrigatório. Contém a carga principal da requisição ou resposta.
Fault	Elemento usado para representar erro SOAP, normalmente dentro do Body.

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <correlacao>abc-123</correlacao>
  </soap:Header>
  <soap:Body>
    <consultarProcessoRequest>
      <numero>5000001-00.2026.8.24.0000</numero>
    </consultarProcessoRequest>
  </soap:Body>
</soap:Envelope>
```

4.2 WSDL - Web Services Description Language

WSDL - Web Services Description Language - é uma linguagem baseada em XML usada para descrever serviços web, especialmente SOAP. Ela funciona como contrato formal: informa operações disponíveis, mensagens, tipos de dados, binding e endereço do serviço. Para prova, associe WSDL a contrato de serviço SOAP, não a documentação informal de endpoint REST.

Pegadinha FGV

SOAP não é sinônimo de WSDL. SOAP é protocolo de mensagens. WSDL é linguagem de descrição de serviço. Eles frequentemente aparecem juntos, mas não são a mesma coisa.

4.3 Quando SOAP tende a ser adequado

SOAP tende a aparecer em integrações com contrato rígido, sistemas legados, interoperabilidade empresarial, padrões WS-* e ambientes nos quais o envelope de mensagem é tão importante quanto o transporte. Em sistemas bancários e públicos, é comum encontrar SOAP em integrações antigas ou altamente reguladas.

5. REST - Representational State Transfer

5.1 REST é estilo arquitetural, não protocolo

REST - Representational State Transfer - é um estilo arquitetural para sistemas distribuídos. A definição clássica vem de Roy Fielding, que descreve REST por restrições arquiteturais, como client-server, stateless, cache, uniform interface, layered system e code-on-demand como restrição opcional. Em prova, dizer que REST é 'protocolo' é, em regra, erro conceitual.

Uma API pode usar HTTP e JSON e ainda assim não respeitar bem REST. O fato de retornar JSON não torna uma API automaticamente RESTful. A prova tende a explorar justamente essa confusão.

Restrição REST	Sentido para prova
Client-server	Cliente e servidor têm responsabilidades separadas. A UI Angular não precisa conhecer detalhes internos do serviço Quarkus.
Stateless	Cada requisição deve conter informações suficientes para ser entendida. O servidor não deve depender de sessão conversacional do cliente.
Cache	Respostas podem ser marcadas como cacheáveis ou não, melhorando desempenho.
Uniform interface	Uso uniforme de recursos, identificadores, representações e métodos padronizados.
Layered system	Cliente pode falar com gateway, proxy ou balanceador sem conhecer a cadeia interna.
Code-on-demand	Opcional. Servidor pode enviar código executável ao cliente, como scripts.

5.2 Recursos e URIs

REST organiza a integração em torno de recursos, não de ações verbais. A URI - Uniform Resource Identifier - deve identificar recursos. Por isso, em desenho REST, costuma-se preferir substantivos no caminho e métodos HTTP para indicar a ação.

```
Menos aderente:
POST /aprovarProcesso
POST /consultarDocumento

Mais aderente:
GET /processos/{numero}
POST /processos/{numero}/aprovacoes
GET /documentos/{id}
```

5.3 Métodos HTTP e semântica

HTTP - Hypertext Transfer Protocol - define métodos com semântica própria. FGV costuma cobrar segurança e idempotência. Método seguro é o método que não pretende alterar estado no servidor. Método idempotente é aquele cujo efeito pretendido de várias requisições idênticas é igual ao efeito de uma única requisição.

Método	Uso típico	Seguro?	Idempotente?	Pegadinha
GET	Obter representação de recurso	Sim	Sim	Não deve alterar estado relevante; não use para aprovar, excluir ou pagar.

POST	Criar subordinado, disparar processamento ou operação não idempotente	Não	Não, em geral	Repetir pode criar duplicidade se não houver controle.
PUT	Criar/substituir completamente recurso em URI conhecida	Não	Sim	Não é atualização parcial por natureza.
PATCH	Alteração parcial de recurso	Não	Não necessariamente	Foi definido para modificação parcial; não confundir com PUT.
DELETE	Remover recurso	Não	Sim, pelo efeito pretendido	A segunda chamada pode retornar 404, mas o efeito pretendido permanece remoção.

5.4 Códigos de status HTTP

Bons contratos de API usam códigos de status de forma coerente. A FGV pode cobrar cenários de retorno. O código não é detalhe decorativo: ele comunica a consequência da requisição.

Código	Nome	Uso típico em APIs
200	OK	Requisição processada com sucesso, geralmente com corpo de resposta.
201	Created	Recurso criado; normalmente acompanhado de Location apontando para o novo recurso.
204	No Content	Sucesso sem corpo de resposta, comum em DELETE ou PUT sem retorno.
400	Bad Request	Requisição malformada ou inválida de modo genérico.
401	Unauthorized	Autenticação ausente ou inválida. Apesar do nome, refere-se a autenticação.
403	Forbidden	Usuário autenticado, mas sem autorização para executar a ação.
404	Not Found	Recurso não encontrado ou não exposto ao cliente.
409	Conflict	Conflito com o estado atual do recurso, como versão divergente ou duplicidade.
422	Unprocessable Content	Sintaxe compreendida, mas semântica inválida; usado em validação de negócio.

6. Desenvolvimento de APIs

6.1 Contrato de API

Contrato de API é a descrição estável do que o consumidor pode esperar: endpoints, métodos, parâmetros, cabeçalhos, corpo de requisição, corpo de resposta, códigos de status, exemplos e erros. Em times modernos, o contrato pode ser escrito antes do código - contract-first - ou derivado do código - code-first.

OpenAPI Specification - OAS - é uma especificação independente de linguagem para descrever APIs HTTP. Ela permite que humanos e máquinas entendam capacidades do serviço sem inspecionar código-fonte, documentação avulsa ou tráfego. Ferramentas como Swagger UI costumam renderizar documentos OpenAPI, mas Swagger UI não é a especificação em si.

```

openapi: 3.1.0
paths:
  /processos/{numero}:
    get:
      summary: Consulta processo
      parameters:
        - name: numero
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Processo encontrado
        '404':
          description: Processo não encontrado

```

6.2 Parâmetros de caminho, query, header e corpo

Local	Exemplo	Uso correto
Path parameter	/processos/{numero}	Identifica recurso específico ou hierarquia de recurso.
Query parameter	/processos?classe=apelação&page=2	Filtragem, ordenação, paginação, busca, projeção.
Header	Authorization, Accept, Content-Type	Metadados da requisição, negociação de conteúdo, autenticação.
Body	JSON/XML com dados	Dados de criação, alteração ou comando complexo.

6.3 JSON, XML e negociação de conteúdo

JSON - JavaScript Object Notation - é formato textual de intercâmbio de dados. Segundo a RFC 8259, um valor JSON pode ser objeto, array, número, string, true, false ou null. XML - Extensible Markup Language - é linguagem de marcação extensível, usada historicamente em SOAP e em integrações corporativas formais.

A negociação de conteúdo usa cabeçalhos como Accept e Content-Type. Content-Type informa o formato do corpo enviado. Accept informa o que o cliente aceita receber. A prova pode trocar esses cabeçalhos.

```
POST /api/v1/processos HTTP/1.1
Content-Type: application/json
Accept: application/json

{
  "numero": "5000001-00.2026.8.24.0000",
  "classe": "Procedimento Comum"
}
```

6.4 Erros de API

Uma boa API não retorna sempre 200 com um campo 'sucesso=false'. Ela usa código HTTP compatível e corpo de erro estável. A RFC 9457, sucessora da RFC 7807, define Problem Details para transportar detalhes de erro de forma legível por máquina em respostas HTTP.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://tjsc.example/problemas/processo-ja-cadastrado",
  "title": "Conflito de cadastro",
  "status": 409,
  "detail": "Já existe processo com este número.",
  "instance": "/processos/5000001-00.2026.8.24.0000"
}
```

6.5 Versionamento e compatibilidade

Versionar API é gerenciar evolução do contrato. Mudanças compatíveis preservam consumidores existentes, como adicionar campo opcional ou novo endpoint. Mudanças incompatíveis quebram consumidores, como renomear campo obrigatório, alterar tipo, remover endpoint ou mudar semântica de status.

Formas comuns de versionamento: no caminho (/v1/processos), em cabeçalho customizado, no media type ou em política explícita de evolução. Para prova, o mais importante é entender que versionamento controla compatibilidade e governança do contrato.

6.6 Idempotência e reprocessamento

Em sistemas corporativos, **falhas de rede e retries são comuns. Idempotência evita efeito duplicado**. Um POST de pagamento, protocolo ou abertura de processo pode exigir chave de idempotência para que repetição acidental não gere duplicidade.

```
POST /pagamentos
Idempotency-Key: 41be0d4b-5f82-4b7e-8f41-abc123

{ "valor": 100.00, "conta": "123" }
```

6.7 API Gateway, observabilidade e governança

API Gateway não é obrigatório para existir API, mas é comum em ambientes corporativos. Pode centralizar roteamento, autenticação, autorização, rate limiting, transformação de payload, auditoria, logs, métricas e políticas. Para a FGV, cuidado: gateway não substitui validação de autorização dentro do serviço quando há regras de negócio finas.

7. SOAP x REST x API

Critério	SOAP	REST
Natureza	Protocolo de mensagens baseado em XML	Estilo arquitetural
Foco	Operações e contrato formal	Recursos e representações
Contrato típico	WSDL	OpenAPI, documentação HTTP, schemas
Formato comum	XML	JSON, XML ou outro formato
Transporte	Pode usar HTTP e outros bindings	Comum sobre HTTP; não se confunde com HTTP
Uso típico	Integrações empresariais formais, legado, WS-*	APIs web, microserviços, integrações leves
Pegadinha	Não usa XSLT como formato de mensagem	Não exige JSON nem CRUD puro

8. Pegadinhas recorrentes no estilo FGV

- SOAP usa XML como base de mensagem; JSON é típico em REST, mas não obrigatório.
- REST é estilo arquitetural; HTTP é protocolo; JSON é formato; API é interface.
- GET é seguro e idempotente; POST geralmente não é idempotente; PUT é idempotente e substitui; PATCH é atualização parcial; DELETE é idempotente pelo efeito pretendido.
- 401 indica problema de autenticação; 403 indica autenticação reconhecida, mas autorização negada.
- Path identifica recurso; query filtra, pagina, ordena ou busca coleções.
- WSDL descreve serviço SOAP; OpenAPI descreve APIs HTTP; Swagger UI é ferramenta, não o contrato em si.
- Stateless não significa sem autenticação; significa que cada requisição deve carregar contexto suficiente, por exemplo token.
- API Gateway ajuda governança, mas não transforma endpoint mal desenhado em RESTful.
- RESTful não é sinônimo de 'usar verbos no path'. Em REST, o verbo de aplicação costuma estar no método HTTP ou em recurso subordinado.
- 201 Created é mais adequado do que 200 OK quando a requisição cria novo recurso; 204 No Content não deve carregar corpo de resposta.

9. Checklist de memorização

- Decorar a cadeia: API é interface; Web Service é serviço via rede; SOAP é protocolo XML; REST é estilo arquitetural; HTTP é protocolo; JSON/XML são formatos.
- Saber explicar Envelope, Header, Body e Fault em SOAP.
- Saber associar WSDL a SOAP e OpenAPI a APIs HTTP.
- Saber diferenciar GET, POST, PUT, PATCH e DELETE por segurança, idempotência e finalidade.
- Saber status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422 e 500.
- Saber desenhar path versus query em endpoints RESTful.
- Saber que stateless não impede autenticação por token.
- Saber que REST não exige JSON, mas JSON é comum em APIs RESTful modernas.

10. Questões autorais no padrão FGV

As questões a seguir são autorais. Não há gabarito nesta seção. Resolva antes de consultar a próxima seção.

Questão 1. Em um projeto de integração entre sistemas de um tribunal e um sistema legado de outra instituição pública, a equipe técnica optou por um serviço que troca mensagens XML estruturadas, com contrato formal de operações, mensagens e tipos de dados. O serviço também prevê campos de cabeçalho para informações de segurança e rastreabilidade. À luz dos padrões de Web Services, a opção que melhor descreve essa solução é:

- A) uma API RESTful, pois todo serviço que utiliza HTTP e XML é REST por definição.
- B) um serviço SOAP, no qual o contrato costuma ser descrito por WSDL e as mensagens são organizadas em envelope XML.
- C) uma API GraphQL, pois o cliente seleciona exatamente os campos que deseja receber.
- D) um serviço RPC informal, pois SOAP não admite cabeçalho de mensagem.
- E) um serviço REST nível 3, pois o uso de XML implica obrigatoriamente hipermídia.

Questão 2. No desenho de uma API para consulta de documentos, a equipe pretende expor a operação de busca de um documento pelo identificador único. Considerando princípios REST e semântica HTTP, a alternativa mais adequada é:

- A) POST /consultarDocumento com corpo contendo o identificador, pois consultas devem sempre usar POST quando houver parâmetro.
- B) GET /documentos/{id}, pois o recurso é identificado na URI e a operação de leitura usa método seguro.
- C) PUT /documentos/{id}, pois PUT é o método padrão para toda operação idempotente.
- D) DELETE /documentos?id={id}, pois a idempotência do DELETE permite consultas sem efeito colateral.
- E) GET /documentos com corpo JSON obrigatório, pois GET deve carregar os filtros no corpo da requisição.

Questão 3. Analise as afirmativas a seguir sobre REST - Representational State Transfer. I. REST é um estilo arquitetural, e não um protocolo de transporte. II. Uma API que retorna JSON é necessariamente RESTful. III. A restrição stateless exige que cada requisição contenha informações suficientes para ser compreendida pelo servidor. Está correto somente o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

Questão 4. Uma equipe implementou endpoint para substituição completa dos dados cadastrais de um recurso em URI conhecida. A operação deve poder ser repetida com o mesmo corpo sem produzir efeito adicional além do mesmo estado final. Nessa situação, o método HTTP mais aderente é:

- A) POST, porque todo envio de corpo deve usar POST.
- B) PATCH, porque toda alteração em recurso existente deve ser parcial.
- C) GET, porque a repetição da chamada precisa ser idempotente.
- D) PUT, porque substitui o recurso identificado e possui semântica idempotente.
- E) OPTIONS, porque descreve as capacidades do recurso e altera seu estado.

Questão 5. A aplicação Angular do usuário autenticado recebe 401 ao consumir uma API interna. Em seguida, após renovar o token de autenticação, a mesma chamada passa a retornar 403. A interpretação mais adequada desses códigos é:

- A) 401 indica problema de autenticação; 403 indica que a autenticação foi reconhecida, mas a autorização foi negada.
- B) 401 indica recurso inexistente; 403 indica erro interno do servidor.
- C) 401 e 403 são equivalentes e podem ser usados indistintamente para qualquer falha de acesso.
- D) 401 indica validação de negócio; 403 indica payload malformatado.
- E) 401 indica conflito de versão do recurso; 403 indica criação bem-sucedida sem corpo.

Questão 6. Em SOAP 1.2, uma mensagem é organizada por elementos definidos em XML. Assinale a alternativa que apresenta corretamente a estrutura conceitual central de uma mensagem SOAP.

- A) Request, Response, Resource e Representation.
- B) Path, Query, Header e Cookie.
- C) Endpoint, Controller, Repository e Entity.
- D) Envelope, Header, Body e Fault.
- E) OpenAPI, Paths, Components e Schemas.

Questão 7. Considere o seguinte trecho simplificado de contrato:

```
openapi: 3.1.0
paths:
  /processos/{numero}:
    get:
      parameters:
        - name: numero
          in: path
          required: true
```

Esse trecho está mais diretamente associado a:

- A) WSDL, usado para descrever operações SOAP por meio de XML.
- B) UDDI, usado para registrar serviços em diretórios públicos.
- C) OpenAPI Specification, usada para descrever APIs HTTP de forma independente de linguagem.
- D) XML Schema exclusivo de SOAP, usado para representar Faults.
- E) Java Persistence API, usada para mapear entidades relacionais.

Questão 8. A respeito de Web Services SOAP, avalie as afirmativas: I. SOAP é baseado em XML. II. SOAP pode ser usado sobre HTTP e, conforme binding, não se limita conceitualmente ao HTTP. III. SOAP utiliza JSON como formato obrigatório de mensagem. Está correto somente o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) III, apenas.
- D) I e III, apenas.
- E) I e II, apenas.

Questão 9. Um serviço RESTful de processos permite listar processos por classe, unidade e situação, com paginação e ordenação. Considerando boas práticas de desenho de API, esses critérios tendem a ser representados como:

- A) segmentos obrigatórios de path, como /processos/classe/civel/unidade/1/situacao/ativo, pois todo critério variável deve estar no caminho.

- B) query parameters, como `/processos?classe=civel&unidade=1&situacao=ativo&page=2`, pois são filtros e controles sobre coleção.
- C) headers obrigatórios, pois filtros de negócio sempre são metadados de transporte.
- D) corpo obrigatório em requisição GET, pois busca complexa não pode usar query string.
- E) códigos de status HTTP, pois filtros devem ser codificados em 2xx ou 4xx.

Questão 10. Uma API de peticionamento eletrônico recebe POST `/peticoes` para criar uma nova petição. A operação é processada com sucesso, gera identificador definitivo e retorna o endereço do novo recurso no cabeçalho Location. O status HTTP mais adequado é:

- A) 201 Created.
- B) 204 No Content, pois toda criação deve omitir corpo e localização.
- C) 304 Not Modified, pois o recurso ainda não foi alterado por clientes.
- D) 400 Bad Request, pois POST nunca deve criar recurso.
- E) 409 Conflict, pois todo novo identificador gera conflito com a coleção.

Questão 11. Relacione os conceitos da Coluna I às descrições da Coluna II.

Coluna I

1. WSDL
2. JSON
3. URI
4. API Gateway

Coluna II

- () Formato textual de intercâmbio de dados com objetos, arrays, strings, números, true, false e null.
- () Identificador de recurso, como `/processos/{numero}`.
- () Componente que pode centralizar roteamento, autenticação, rate limiting e políticas de acesso.
- () Linguagem de descrição associada a serviços web, especialmente SOAP.

A sequência correta é:

- A) 1 - 2 - 3 - 4.
- B) 2 - 3 - 4 - 1.
- C) 3 - 2 - 1 - 4.
- D) 4 - 1 - 3 - 2.
- E) 2 - 4 - 3 - 1.

Questão 12. Sobre idempotência em APIs HTTP, assinale a afirmativa correta.

- A) POST é sempre idempotente porque cria recurso em coleção.
- B) PATCH é necessariamente idempotente porque altera apenas parte de um recurso.
- C) DELETE é considerado idempotente quanto ao efeito pretendido, ainda que uma segunda chamada possa ter resposta diferente.
- D) GET não é idempotente porque pode ser chamado muitas vezes.
- E) PUT não é idempotente porque substitui o recurso inteiro.

Questão 13. Uma equipe decidiu retornar sempre HTTP 200 OK, tanto em sucesso quanto em erro, informando no corpo JSON um campo `sucesso=true` ou `sucesso=false`. Para uma API HTTP bem desenhada, a crítica tecnicamente mais adequada é:

- A) A estratégia é obrigatória em APIs RESTful, pois códigos 4xx e 5xx pertencem apenas a navegadores.
- B) A estratégia é adequada apenas se o payload estiver em XML, pois JSON não admite erros estruturados.
- C) A estratégia impede o uso de GET, mas não afeta consumidores POST.
- D) A estratégia é irrelevante, pois clientes nunca interpretam status HTTP.
- E) A estratégia empobrece o contrato, pois códigos HTTP devem expressar a classe do resultado e o corpo pode detalhar o erro.

Questão 14. A equipe de desenvolvimento precisa expor endpoint para alteração apenas do e-mail de uma pessoa, sem exigir o reenvio de todos os demais campos cadastrais. Segundo a semântica HTTP padronizada, o método mais adequado é:

- A) PUT, pois qualquer alteração parcial deve substituir integralmente o recurso.
- B) PATCH, pois foi definido para aplicar modificações parciais a um recurso.
- C) GET, pois alteração de campo simples pode ser segura.
- D) HEAD, pois cabeçalho HTTP é suficiente para alterar metadados de recurso.
- E) TRACE, pois permite refletir o conteúdo da requisição e aplicar a mudança.

Questão 15. Em relação a Content-Type e Accept em APIs HTTP, assinale a alternativa correta.

- A) Content-Type informa o tipo de mídia do corpo enviado; Accept informa os tipos de resposta aceitos pelo cliente.
- B) Content-Type informa autorização; Accept informa autenticação.
- C) Content-Type e Accept são equivalentes e indicam sempre o formato da URL.
- D) Accept deve ser usado apenas em SOAP, pois REST não negocia conteúdo.
- E) Content-Type informa o método HTTP que será executado.

Questão 16. Um serviço interno expõe POST /processos/{numero}/arquivamentos para registrar o arquivamento de um processo, gerando registro de evento. Embora a palavra 'arquivamento' pareça ação, a modelagem pode ser considerada aceitável em REST quando:

- A) o endpoint representa um recurso subordinado ou coleção de eventos de arquivamento, e o POST cria uma nova ocorrência nessa coleção.
- B) a URI contém qualquer verbo em português, pois REST exige verbos no caminho.
- C) o serviço retorna XML, pois XML transforma ações em recursos.
- D) o endpoint é chamado por Angular, pois REST depende do tipo de cliente.
- E) o método usado é POST, pois POST torna qualquer URI automaticamente RESTful.

Questão 17. Assinale a alternativa que melhor diferencia SOAP e REST no contexto de integração entre sistemas.

- A) SOAP é um formato de banco de dados; REST é uma linguagem de programação.
- B) SOAP exige exclusivamente transporte SMTP; REST exige exclusivamente XML.
- C) SOAP é usado apenas no front-end; REST é usado apenas em banco de dados.
- D) SOAP e REST são sinônimos quando a API usa HTTP.
- E) SOAP é protocolo de mensagens baseado em XML; REST é estilo arquitetural orientado a recursos e representações.

Questão 18. No desenvolvimento de APIs para sistemas corporativos, versionamento de contrato é relevante principalmente porque:

- A) elimina a necessidade de autenticação e autorização.
- B) permite evoluir a API controlando compatibilidade com consumidores existentes.
- C) transforma automaticamente APIs SOAP em RESTful.
- D) impede que a API utilize status HTTP 4xx.
- E) substitui documentação, testes e observabilidade.

Questão 19. Uma API possui o endpoint GET /usuarios/{id}. A aplicação cliente permite que o usuário informe manualmente o id e consegue acessar dados de outros usuários apenas alterando esse identificador. Do ponto de vista de desenho e segurança de APIs, o problema central é:

- A) uso de GET, que é sempre inseguro e proibido em APIs autenticadas.
- B) ausência de XML no corpo da requisição.
- C) falha de autorização no nível do objeto, pois o serviço não valida se o usuário pode acessar o recurso identificado.
- D) uso de path parameter, que é incompatível com REST.
- E) uso de JSON, que impede controle de acesso por recurso.

Questão 20. Um time deseja gerar documentação interativa e clientes de consumo a partir de um contrato que descreva endpoints, parâmetros, corpos, respostas e esquemas de uma API HTTP. A escolha mais aderente a esse objetivo é:

- A) WSDL 1.1, por ser o formato nativo para APIs RESTful baseadas em JSON.
- B) UDDI, por substituir os métodos HTTP por operações XML.
- C) SOAP Envelope, por documentar path parameters e responses HTTP.
- D) OpenAPI Specification, por descrever APIs HTTP de modo estruturado e independente de linguagem.
- E) JPA, por definir os códigos de status de uma API.

11. Gabarito resumido

1-B; 2-B; 3-C; 4-D; 5-A; 6-D; 7-C; 8-E; 9-B; 10-A; 11-B; 12-C; 13-E; 14-B; 15-A; 16-A; 17-E; 18-B; 19-C; 20-D.

12. Comentários completos

Questão 1 - Gabarito: B

O cenário descreve SOAP: mensagens XML, cabeçalhos e contrato formal de operações, tipicamente WSDL.

Alternativa A: incorreta. REST não decorre apenas de usar HTTP ou XML; REST é estilo baseado em recursos e restrições.

Alternativa B: correta. O cenário descreve SOAP: mensagens XML, cabeçalhos e contrato formal de operações, tipicamente WSDL.

Alternativa C: incorreta. GraphQL não aparece no cenário e teria consulta flexível de campos pelo cliente.

Alternativa D: incorreta. SOAP admite Header; logo a alternativa nega característica essencial.

Alternativa E: incorreta. XML não implica hipermídia nem nível 3 de maturidade REST.

Questão 2 - Gabarito: B

GET /documentos/{id} identifica um recurso e usa método seguro para leitura.

Alternativa A: incorreta. POST pode transportar parâmetro, mas não é a escolha natural para leitura simples por identificador.

Alternativa B: correta. GET /documentos/{id} identifica um recurso e usa método seguro para leitura.

Alternativa C: incorreta. PUT é idempotente, mas sua semântica é substituir/criar recurso, não consultar.

Alternativa D: incorreta. DELETE remove recurso; sua idempotência não o torna método de consulta.

Alternativa E: incorreta. GET com corpo obrigatório é desenho frágil e pouco interoperável; filtros simples vão em path/query.

Questão 3 - Gabarito: C

I e III estão corretas: REST é estilo arquitetural e stateless exige que cada requisição carregue contexto suficiente.

Alternativa A: incorreta. Ignora a afirmativa III, que está correta.

Alternativa B: incorreta. II está errada: JSON é formato comum, não condição suficiente para REST.

Alternativa C: correta. I e III estão corretas: REST é estilo arquitetural e stateless exige que cada requisição carregue contexto suficiente.

Alternativa D: incorreta. Inclui II, que confunde formato com estilo arquitetural.

Alternativa E: incorreta. Inclui II; uma API JSON pode ser RPC sobre HTTP e não RESTful.

Questão 4 - Gabarito: D

PUT é o método com semântica de substituição do recurso identificado e é idempotente.

Alternativa A: incorreta. POST normalmente cria subordinado ou dispara processamento, não substitui recurso em URI conhecida.

Alternativa B: incorreta. PATCH é parcial; o enunciado fala em substituição completa.

Alternativa C: incorreta. GET é leitura segura, não alteração.

Alternativa D: correta. PUT é o método com semântica de substituição do recurso identificado e é idempotente.

Alternativa E: incorreta. OPTIONS consulta capacidades, não altera estado de recurso.

Questão 5 - Gabarito: A

401 se relaciona à autenticação; 403 indica que o usuário autenticado não tem autorização para a ação.

Alternativa A: correta. 401 se relaciona à autenticação; 403 indica que o usuário autenticado não tem autorização para a ação.

Alternativa B: incorreta. 404 seria recurso não encontrado; 500 seria erro interno.

Alternativa C: incorreta. 401 e 403 não são equivalentes.

Alternativa D: incorreta. Validação de negócio costuma aparecer como 400/422, não 401/403.

Alternativa E: incorreta. Conflito é 409; criação sem corpo poderia ser 201/204 conforme contexto.

Questão 6 - Gabarito: D

A estrutura central SOAP inclui Envelope, Header, Body e Fault.

Alternativa A: incorreta. Mistura conceitos de REST: recurso e representação.

Alternativa B: incorreta. São locais ou elementos de HTTP, não estrutura SOAP.

Alternativa C: incorreta. São camadas/classes típicas de aplicação, não mensagem SOAP.

Alternativa D: correta. A estrutura central SOAP inclui Envelope, Header, Body e Fault.

Alternativa E: incorreta. São partes de OpenAPI, não de SOAP.

Questão 7 - Gabarito: C

O trecho é OpenAPI 3.1.0: descreve paths, operações, parâmetros e respostas de uma API HTTP.

Alternativa A: incorreta. WSDL é XML e mais associado a SOAP, não a essa estrutura YAML/JSON de OpenAPI.

Alternativa B: incorreta. UDDI é diretório/registro de serviços, não contrato de endpoint HTTP.

Alternativa C: correta. O trecho é OpenAPI 3.1.0: descreve paths, operações, parâmetros e respostas de uma API HTTP.

Alternativa D: incorreta. Não é XML Schema nem SOAP Fault.

Alternativa E: incorreta. JPA trata persistência em Java, não contrato de API.

Questão 8 - Gabarito: E

SOAP é baseado em XML e pode usar HTTP ou outros bindings. Afirmativa III está errada porque JSON não é obrigatório em SOAP.

Alternativa A: incorreta. Deixa de incluir II, que está correta.

Alternativa B: incorreta. Deixa de incluir I, que está correta.

Alternativa C: incorreta. III é falsa.

Alternativa D: incorreta. Inclui III, falsa, e exclui II, verdadeira.

Alternativa E: correta. SOAP é baseado em XML e pode usar HTTP ou outros bindings. Afirmativa III está errada porque JSON não é obrigatório em SOAP.

Questão 9 - Gabarito: B

Filtros, paginação e ordenação sobre uma coleção são naturalmente query parameters.

Alternativa A: incorreta. Path deve representar identidade/hierarquia de recurso; usar tudo como path reduz flexibilidade.

Alternativa B: correta. Filtros, paginação e ordenação sobre uma coleção são naturalmente query parameters.

Alternativa C: incorreta. Headers são metadados, não local típico de filtros de negócio.

Alternativa D: incorreta. GET com corpo obrigatório tem baixa interoperabilidade.

Alternativa E: incorreta. Status codes comunicam resultado, não critérios de busca.

Questão 10 - Gabarito: A

201 Created é o status adequado para criação de recurso; o cabeçalho Location pode indicar a URI do novo recurso.

Alternativa A: correta. 201 Created é o status adequado para criação de recurso; o cabeçalho Location pode indicar a URI do novo recurso.

Alternativa B: incorreta. 204 indica sucesso sem conteúdo, mas não expressa tão bem criação com Location quanto 201.

Alternativa C: incorreta. 304 relaciona-se a cache/condicional, não criação.

Alternativa D: incorreta. POST pode criar recurso.

Alternativa E: incorreta. 409 é conflito, não sucesso.

Questão 11 - Gabarito: B

JSON corresponde à descrição 2; URI à 3; API Gateway à 4; WSDL à 1.

Alternativa A: incorreta. Coloca WSDL como JSON, erro de categoria.

Alternativa B: correta. JSON corresponde à descrição 2; URI à 3; API Gateway à 4; WSDL à 1.

Alternativa C: incorreta. Começa com URI como JSON.

Alternativa D: incorreta. Começa com gateway como JSON.

Alternativa E: incorreta. Troca URI e gateway no meio da sequência.

Questão 12 - Gabarito: C

DELETE é idempotente quanto ao efeito pretendido: após uma ou várias chamadas, o recurso deve permanecer removido, ainda que respostas possam diferir.

Alternativa A: incorreta. POST não é sempre idempotente; pode criar duplicidade.

Alternativa B: incorreta. PATCH pode ser idempotente em alguns desenhos, mas não necessariamente.

Alternativa C: correta. DELETE é idempotente quanto ao efeito pretendido: após uma ou várias chamadas, o recurso deve permanecer removido, ainda que respostas possam diferir.

Alternativa D: incorreta. GET é idempotente e seguro.

Alternativa E: incorreta. PUT é idempotente por semântica.

Questão 13 - Gabarito: E

O contrato fica mais claro quando status HTTP comunica a classe do resultado e o corpo detalha o problema.

Alternativa A: incorreta. Códigos 4xx/5xx são essenciais em APIs HTTP, não apenas em navegadores.

Alternativa B: incorreta. JSON admite erro estruturado; inclusive application/problem+json.

Alternativa C: incorreta. O problema afeta qualquer método, não apenas GET.

Alternativa D: incorreta. Clientes, gateways, logs e ferramentas interpretam status HTTP.

Alternativa E: correta. O contrato fica mais claro quando status HTTP comunica a classe do resultado e o corpo detalha o problema.

Questão 14 - Gabarito: B

PATCH foi definido para aplicar modificações parciais a recursos.

Alternativa A: incorreta. PUT tende à substituição completa, não alteração parcial.

Alternativa B: correta. PATCH foi definido para aplicar modificações parciais a recursos.

Alternativa C: incorreta. GET é seguro e não deve alterar estado.

Alternativa D: incorreta. HEAD obtém metadados de resposta, não altera recurso.

Alternativa E: incorreta. TRACE é diagnóstico, não atualização.

Questão 15 - Gabarito: A

Content-Type descreve o tipo de mídia do corpo enviado; Accept descreve o que o cliente aceita receber.

Alternativa A: correta. Content-Type descreve o tipo de mídia do corpo enviado; Accept descreve o que o cliente aceita receber.

Alternativa B: incorreta. Autenticação/autorização usam cabeçalhos como Authorization, não Content-Type/Accept.

Alternativa C: incorreta. Não são equivalentes.

Alternativa D: incorreta. REST/HTTP também usa negociação de conteúdo.

Alternativa E: incorreta. Método HTTP é GET/POST/PUT etc.; não é informado por Content-Type.

Questão 16 - Gabarito: A

Modelar arquivamentos como coleção subordinada torna o POST criação de um recurso-evento, o que pode ser aderente a REST.

Alternativa A: correta. Modelar arquivamentos como coleção subordinada torna o POST criação de um recurso-evento, o que pode ser aderente a REST.

Alternativa B: incorreta. REST não exige verbos no caminho; prefere recursos.

Alternativa C: incorreta. XML não transforma ações em recursos.

Alternativa D: incorreta. O tipo de cliente não define REST.

Alternativa E: incorreta. POST sozinho não torna qualquer desenho RESTful.

Questão 17 - Gabarito: E

SOAP é protocolo de mensagens XML; REST é estilo arquitetural orientado a recursos e representações.

Alternativa A: incorreta. Nenhum dos dois é linguagem ou formato de banco de dados.

Alternativa B: incorreta. SOAP não exige exclusivamente SMTP; REST não exige XML.

Alternativa C: incorreta. Ambos podem ser usados no back-end e integração entre sistemas.

Alternativa D: incorreta. Usar HTTP não torna SOAP e REST sinônimos.

Alternativa E: correta. SOAP é protocolo de mensagens XML; REST é estilo arquitetural orientado a recursos e representações.

Questão 18 - Gabarito: B

Versionamento controla evolução do contrato e reduz quebra de consumidores.

Alternativa A: incorreta. Versionamento não elimina autenticação/autorização.

Alternativa B: correta. Versionamento controla evolução do contrato e reduz quebra de consumidores.

Alternativa C: incorreta. Não converte SOAP em REST.

Alternativa D: incorreta. Não impede uso de códigos 4xx.

Alternativa E: incorreta. Não substitui documentação, testes nem observabilidade.

Questão 19 - Gabarito: C

O caso descreve Broken Object Level Authorization: o serviço aceita o id informado sem validar se o usuário tem permissão sobre aquele objeto.

Alternativa A: incorreta. GET é apropriado para leitura; o problema é autorização, não o método em si.

Alternativa B: incorreta. XML não resolve controle de acesso.

Alternativa C: correta. O caso descreve Broken Object Level Authorization: o serviço aceita o id informado sem validar se o usuário tem permissão sobre aquele objeto.

Alternativa D: incorreta. Path parameter é compatível com REST.

Alternativa E: incorreta. JSON não impede autorização por recurso.

Questão 20 - Gabarito: D

OpenAPI descreve endpoints, parâmetros, corpos, respostas e schemas de APIs HTTP, podendo alimentar documentação e geração de clientes.

Alternativa A: incorreta. WSDL é típico de SOAP, não formato nativo de REST/JSON.

Alternativa B: incorreta. UDDI é registro/descoberta, não substitui métodos HTTP.

Alternativa C: incorreta. SOAP Envelope transporta mensagem, não documenta paths e responses HTTP.

Alternativa D: correta. OpenAPI descreve endpoints, parâmetros, corpos, respostas e schemas de APIs HTTP, podendo alimentar documentação e geração de clientes.

Alternativa E: incorreta. JPA é persistência Java, não especificação de API HTTP.

13. Referências consultadas

- Edital retificado TJSC 2026 - Analista de Sistemas - FGV.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV - provas e editais correlatos de Analista de Sistemas, Web Services, SOAP, REST e APIs.
- W3C - SOAP Version 1.2 Part 1: Messaging Framework; SOAP Version 1.2 Part 2: Adjuncts.
- IETF RFC 9110 - HTTP Semantics.
- IETF RFC 5789 - PATCH Method for HTTP.
- IETF RFC 8259 - The JavaScript Object Notation (JSON) Data Interchange Format.
- IETF RFC 9457 - Problem Details for HTTP APIs.
- OpenAPI Initiative - OpenAPI Specification 3.1.0.
- Roy T. Fielding - Architectural Styles and the Design of Network-based Software Architectures, Chapter 5: REST.
- OWASP API Security Top 10 2023, usado apenas como apoio para o exemplo de autorização por objeto em APIs.